



Objective & Lecture Mapping: In previous labs, we built a Simple Reflex Agent that moved randomly or reacted only to immediate adjacent cells. Today, we transition to a **Goal-Based/Planning Agent**. You will expose the environment's state space to the agent, allowing it to use **Breadth-First Search (BFS)**, **Depth-First Search (DFS)**, and **Uniform-Cost Search (UCS)** to simulate and find paths to the food before taking a physical action.

Part 1: Practical Implementation — Building the Search Agent

Step 1.1: Exposing the World Model

Classical search algorithms require an abstract model of the environment to simulate future states. Currently, your agent is "blind" to the overall map.

1. Create a new Branch called Week_03 in your Forked Github Repo and work on the below Questions.
2. Open `visual_grid_game.py`.
3. Locate the `get_percept(self)` method.
4. Add three new keys to the dictionary to pass the global state to the agent:

```
'grid_size': (self.width, self.height)
'walls': list(self.walls)
'all_food': list(self.food_positions)
```

Step 1.2: Implementing BFS, DFS, and UCS

You will implement three distinct uninformed search strategies. The core node expansion structure is identical; only the data structure for the **Frontier** changes!

1. Open `agent.py` and create a new class called `SearchAgent`.
2. Import required structures: `from collections import deque` and `import heapq`.
3. **BFS:** Create `def bfs_search(...)`. Use a FIFO queue (`deque.popleft()`). This explores the shallowest nodes first.
4. **DFS:** Create `def dfs_search(...)`. Use a LIFO stack (`list.pop()`). This explores the deepest nodes first.
5. **UCS:** Create `def ucs_search(...)`. Use a Priority Queue (`heapq.heappop()`) ordered by the total path cost $g(n)$.
6. For all three algorithms, you must maintain a `reached` set (or visited array) to track explored states. This converts your algorithm from a *Tree Search* to a *Graph Search*, preventing infinite loops.

Step 1.3: Executing and Comparing Offline Plans

The agent must now form a complete plan and execute it step-by-step.

1. In your `SearchAgent.__init__`, add an empty list: `self.plan = []` and a config string `self.active_algo = 'BFS'`.
2. In `sense_and_act(self, percept)`, check if `self.plan` is empty.
3. If empty, find the closest food pellet from `percept['all_food']`, execute the search method matching `self.active_algo`, and store the resulting sequence of actions in `self.plan`.
4. Return the first action from the plan using `return self.plan.pop(0)`.

5. **Observation Task:** Change `self.active_algo` between 'BFS', 'DFS', and 'UCS' and run the simulation. Watch how DFS takes erratic, winding paths, while BFS and UCS take direct, optimal paths!

Part 2: Theoretical Evaluation

Based on your implementation and Lecture 04, complete the following analytical questions.

1. (Remember) You built your search logic based on the environment coordinates. What is the fundamental difference between the "State Space" and the "Search Tree"?

State Space: The underlying graph representing all possible valid configurations and the valid action transitions between them. It is static, defined entirely by the environment formulation, and finite for a bounded grid.
Search Tree: A dynamic tree structure constructed during search execution to simulate paths through the state space. The root node is the initial state, and branches represent sequences of actions. Search tree nodes store path metadata, meaning a single physical state coordinate can appear multiple times across different nodes in the tree.

2. (Understand) In Step 1.2, you were instructed to use a 'reached' set. In graph search theory, what specific problem does this set solve, and what would happen to your DFS agent without it?

Problem Solved: The reached set converts a standard Tree Search into a Graph Search by keeping track of every unique state coordinate already expanded or generated. This eliminates redundant paths and breaks infinite loops caused by cyclic trajectories in the grid.
Impact on DFS: Without a reached set, a DFS agent exploring a grid with reversible actions will fall into an infinite loop. Because DFS always expands the deepest node first, it will oscillate infinitely between adjacent coordinates along an unbounded branch without ever reaching the goal or terminating.

3. (Analyze) Compare the visual paths generated by BFS and DFS in your simulation. Why is BFS guaranteed to be optimal in this specific grid, whereas DFS produces winding, suboptimal routes?

BFS Optimality: BFS expands nodes level-by-level in increasing order of depth using a FIFO queue. In a standard grid where every movement step has a uniform cost, the path cost is directly equal to the node's depth. Consequently, the first goal node BFS encounters is guaranteed to be at the shallowest depth, yielding an optimal shortest path.
DFS Suboptimality: DFS follows a LIFO strategy, diving as deeply as possible along a single branch before backtracking. It arbitrarily commits to its initial direction, wandering through long, erratic paths without regard for path cost. It returns the first goal path it stumbles across, which is frequently long and suboptimal.

4. (Evaluate) If we scaled 'visual_grid_game.py' up to a massive 1000x1000 grid, BFS and UCS might crash before finding food. According to the time and space complexity formulas from the lecture, contrast the memory bottlenecks of BFS versus DFS.

BFS Memory Bottleneck: BFS has an exponential space complexity. Because BFS must retain every generated node in memory across the expanding frontier at the current level, scaling to a massive grid causes the memory requirements to explode exponentially, leading to out-of-memory crashes long before deep goals are found.
DFS Memory Efficiency: DFS has a linear space complexity. At any point during its search, DFS only needs to store the single active path from the root node down to the current leaf node, along with remaining unexpanded sibling nodes at each level. This linear memory footprint prevents memory exhaustion, allowing DFS to run without crashing on large grids.

Once Completed Get a PDF of the completed File and Push into the Week 03 brach in the Github Project

Finalize and Lock Lab 03 Documentation

